Internship Report

Design and Implementation of an Internal Helpdesk Chatbot

Orange Morocco - MEDITELECOM

Student	Aksikas Mohamed Souhaib
Host company	MEDITELECOM - Orange Morocco
Internship tutor	Rachid Dinar
Academic supervisor	ZAID HADACH
Period	From 01/06/2026 to 30/07/2026
Duration	2 months
Level	First year of the engineering cycle

Academic year 2025-2026

Contents

1 Abstract 2

2 Introduction 2

3 Internship Context 2

4 Project Objectives, Scope, and Needs 3

5 Work Methodology 4

6 General Architecture 4

7 Chatbot Workflow Diagram 5

8 User Interface 5

9 Knowledge Base 6

10 Administration Area and Knowledge Creation 6

11 Visual Guides: Concept 8

12 Visual Guides: Admin Workflow 8

13 Visual Guide Examples 8

14 Artificial Intelligence Role 10

15 Voice, Image, and Email Assistance 10

16 Security and Confidentiality 11

17 Testing and Validation 11

18 Detailed Technical Stack and Justification 12

19 In-Depth System Architecture 13

20 Database Design and Schema 14

21 Frontend and Administration Mechanics 14

22 Deployment and Infrastructure 15

23 Security and Maintenance 16

1 Abstract

This report presents the work carried out during a two-month internship at MEDITELECOM - Orange Morocco. The project consisted of designing and implementing an internal helpdesk chatbot for collaborators. Its objective was to improve access to support information, reduce repetitive hotline requests, and help users solve common incidents faster.

The developed solution is a web application composed of a chat interface, a Python backend, a knowledge base, an administration area, and artificial intelligence services. It supports text questions, voice messages, image analysis, visual guides, and HelpDesk email generation. The chatbot is not designed to replace human support. It acts as a first-level assistant that answers frequent questions and prepares a clearer escalation when the issue requires human intervention.

The project also includes an important administration dimension. A chatbot becomes useful only if its content remains accurate. For this reason, the administrator can add procedures, attach PDF documents, upload guide screenshots, structure text, monitor conversations, and improve weak answers based on feedback.

The main result is a functional prototype that demonstrates how an internal chatbot can combine validated knowledge and AI capabilities. The knowledge base provides reliable internal context, while AI helps with reformulation, image interpretation, voice transcription, and email writing. This combination makes the solution practical, flexible, and suitable for progressive improvement.

2 Introduction

Internal collaborators often face small but blocking technical issues: access problems, email configuration, network connection, VPN settings, software usage, password procedures, or file-transfer steps. These issues may be simple, but they consume time when the information is scattered across documents, emails, or support channels.

In many cases, the same question is asked several times by different users. A hotline agent may repeat the same procedure, while the user waits for a response that could have been automated. This creates unnecessary pressure on the support team and slows down the resolution of more complex incidents.

The chatbot project addresses this situation by offering a single conversational entry point. Instead of searching for a document or calling support immediately, the user can ask a question in natural language. The chatbot searches the knowledge base, prepares a structured answer, and displays the relevant steps. When the answer requires images, the chatbot can display a visual guide directly in the conversation.

This report explains the internship context, needs analysis, solution design, technical workflow, administration features, visual guide mechanism, use of AI, validation process, and main perspectives for improvement. The focus is both functional and technical, because the value of this project comes from the connection between user experience, backend logic, knowledge quality, and maintainability.

3 Internship Context

The internship took place at MEDITELECOM, commercially known as Orange Morocco. The work environment is linked to digital services, internal information systems, and assistance for collaborators. In such an environment, the availability of IT tools has a direct impact on daily productivity.

Orange collaborators use many internal services: email, network access, VPN, identity management, office tools, file-transfer platforms, and business applications. When one of these tools is not configured correctly, the user may lose time before identifying the right procedure. A chatbot can reduce that friction by centralizing answers in a single interface.

The internship was supervised within the company by Mr. Rachid Dinar and academically followed by Mr. ZAID HADACH. The supervision helped define the project priorities, validate the expected features, and keep the prototype aligned with the internship objectives.

The project was developed in a realistic professional context. It had to be simple enough to be tested quickly, but structured enough to evolve later. This required thinking about deployment, administration, security, user adoption, and future integration with internal tools.

The chosen format, a web chatbot, fits this context well. Users do not need to install a desktop application. Administrators can update the knowledge base from a browser. The backend remains responsible for processing, security checks, database access, and calls to AI services.

4 Project Objectives, Scope, and Needs

The main objective was to design and implement a chatbot that assists internal collaborators with common support requests. The solution had to be useful for end users while remaining maintainable for administrators.

The operational objectives were:

- improve the digital experience of internal collaborators;
- increase user autonomy when handling simple incidents;
- reduce repetitive calls and messages sent to the hotline;
- centralize internal procedures in a searchable knowledge base;
- support visual procedures with screenshots;
- provide voice, image, and email assistance when needed.

The chatbot scope was intentionally focused on first-level support. It should answer frequent questions, explain procedures, and help users collect the right information. It should not make sensitive decisions, access confidential systems without control, or replace specialized technical teams for complex incidents.

This scope is important because it keeps the project realistic. A good internal assistant must be reliable in a defined perimeter before becoming broader. The first version therefore focuses on the most useful workflows: ask a question, retrieve a procedure, display visual help, and escalate clearly when necessary.

The project also aimed to create a foundation for continuous improvement. Each conversation can reveal missing content, unclear procedures, or repeated incidents. The administrator can then enrich the knowledge base and improve future answers.

Needs analysis

The chatbot must serve two profiles. The internal collaborator wants a quick answer, written in simple language, with concrete steps. The administrator needs tools to add content, update procedures, consult conversations, and monitor response quality.

For collaborators, the main expectation is speed and clarity. For administrators, the main expectation is maintainability. The administration area must support procedure creation, PDF upload, image upload, visual-guide preparation, conversation review, and feedback analysis.

The needs can be grouped into four categories:

- direct assistance through text answers;
- multimodal assistance through voice and images;
- knowledge management through an admin interface;
- continuous improvement through history and feedback.

The project therefore goes beyond a simple AI chat. It is a support system built around content, workflow, and supervision.

5 Work Methodology

The work was organized through short iterations. The first phase focused on understanding the problem and identifying the essential user journeys. The second phase focused on building a simple working version: a web interface, a backend, and a first knowledge base.

After this first version, additional features were added gradually. The administration area made it possible to maintain the content. The visual-guide feature allowed the chatbot to return screenshots with explanations. Voice transcription and image analysis improved the flexibility of user input. HelpDesk email generation supported escalation.

Each iteration followed a practical validation logic. A feature was not considered useful only because it existed in the code. It had to support a real scenario: a user asks a question, the backend processes it, the knowledge base provides context, and the interface displays a clear response.

The methodology also emphasized separation of responsibilities. The frontend manages interaction and display. The backend manages routing, processing, security, database access, and AI calls. The knowledge base stores internal procedures. This separation makes the project easier to understand, test, and extend.

This approach was appropriate for the internship duration. It made it possible to deliver a functional prototype while keeping enough flexibility to improve the design as new needs appeared.

6 General Architecture

The solution is composed of several connected parts. The user interacts with the web interface. The interface sends requests to the Python backend. The backend searches the knowledge base, stores useful events, and calls AI services only when needed.

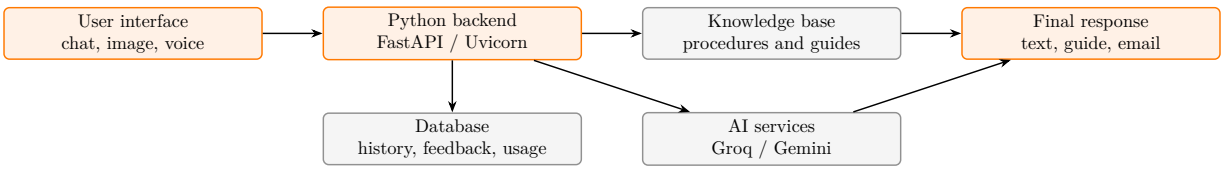


Figure 1: General architecture of the chatbot solution

The backend plays the central orchestration role. It validates the request, prepares the context, selects the useful sources, and returns a safe response. Keeping this logic on the server avoids exposing sensitive rules in the browser.

The knowledge base is the preferred source of information. AI is used as a complement: it improves formulation, analyzes screenshots, transcribes voice, or helps produce an email. This architecture keeps the chatbot aligned with internal procedures instead of relying on generic answers.

7 Chatbot Workflow Diagram

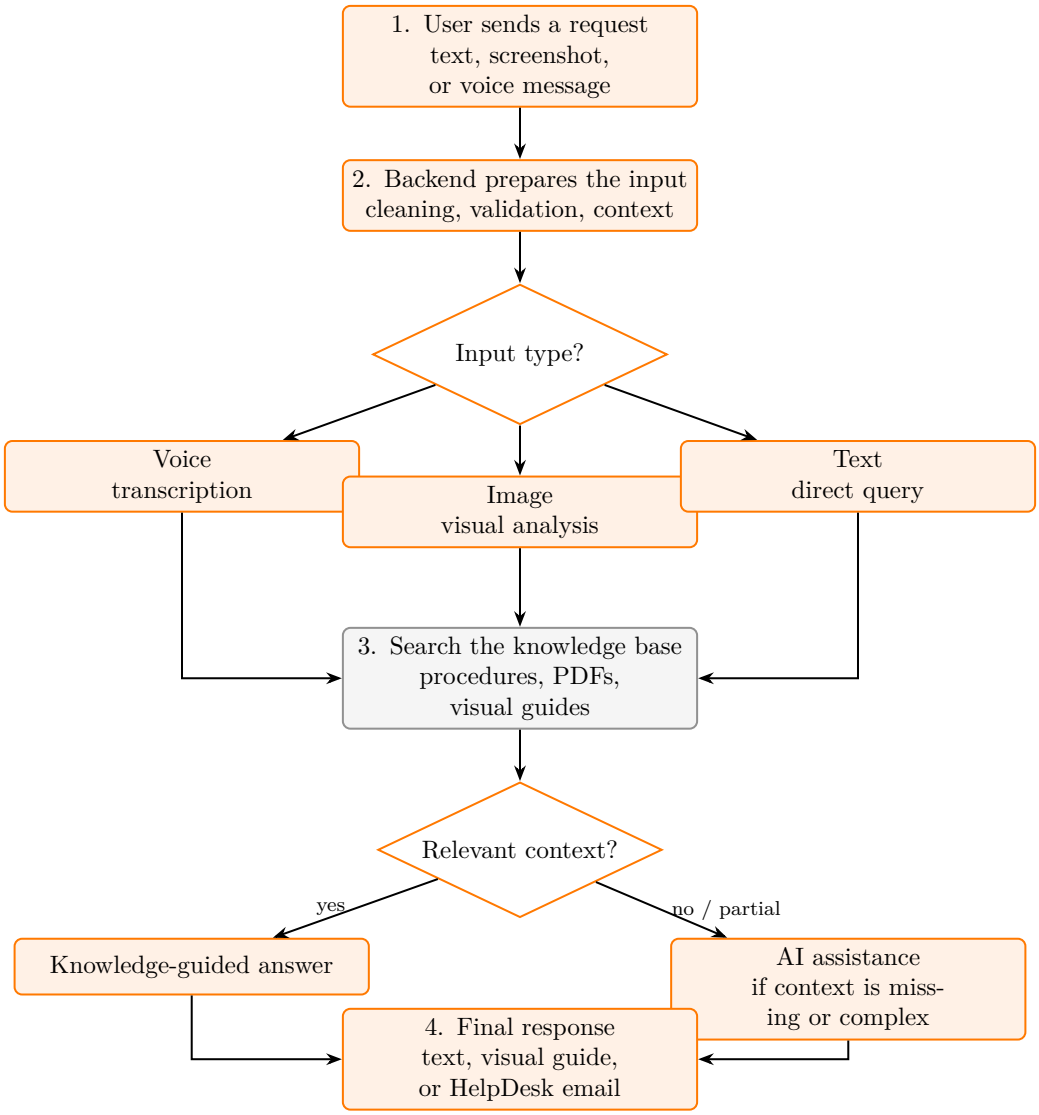


Figure 2: Chatbot workflow diagram

The diagram shows that the chatbot does not immediately send every request to an AI model. It first prepares the request and searches internal knowledge. AI is called when it adds value: transcription, image analysis, reformulation, or completion of an answer. The final response is displayed in the most useful format.

8 User Interface

The user interface is built as a conversation. This design is familiar and reduces the learning effort. The collaborator can type a question, send an image, record a voice message, or continue with a follow-up question.

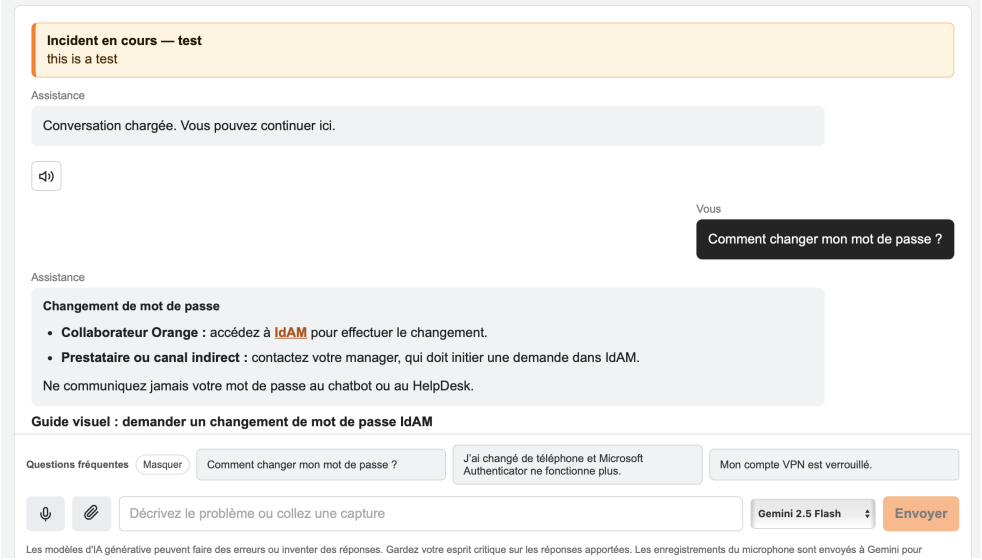


Figure 3: Chatbot user interface

The interface must remain simple because the user usually opens it while facing a problem. The main goal is not to impress the user with technical options, but to guide them quickly. The response must be readable, structured, and directly actionable.

Conversation continuity is also important. If the first answer is not enough, the user can ask a second question without restarting the context. This makes the chatbot closer to a support assistant than a static FAQ page.

9 Knowledge Base

The knowledge base is the most important functional component of the chatbot. It stores internal procedures, validated answers, PDF content, and visual guides. When a request arrives, the backend searches for relevant knowledge before relying on generative AI.

A strong knowledge entry must contain a clear title, useful keywords, a structured description, and simple steps. If needed, it can also include screenshots. This structure helps the chatbot find the right content and produce a precise answer.

The knowledge base also improves consistency. If several users ask the same question, the chatbot can return the same validated procedure. This reduces variation in answers and prevents users from receiving contradictory instructions.

Maintaining the knowledge base is an ongoing task. Procedures can change, screenshots can become outdated, and new incidents can appear. The administration interface therefore plays a key role in keeping the chatbot useful over time.

In practice, the knowledge base transforms individual support knowledge into shared reusable content. Instead of answering the same request repeatedly, the support team can document it once and let the chatbot reuse it.

10 Administration Area and Knowledge Creation

The administration area allows the chatbot to evolve without changing the source code. The administrator can add new knowledge, edit existing content, attach files, add visual guides, review conversation history, and analyze user feedback.

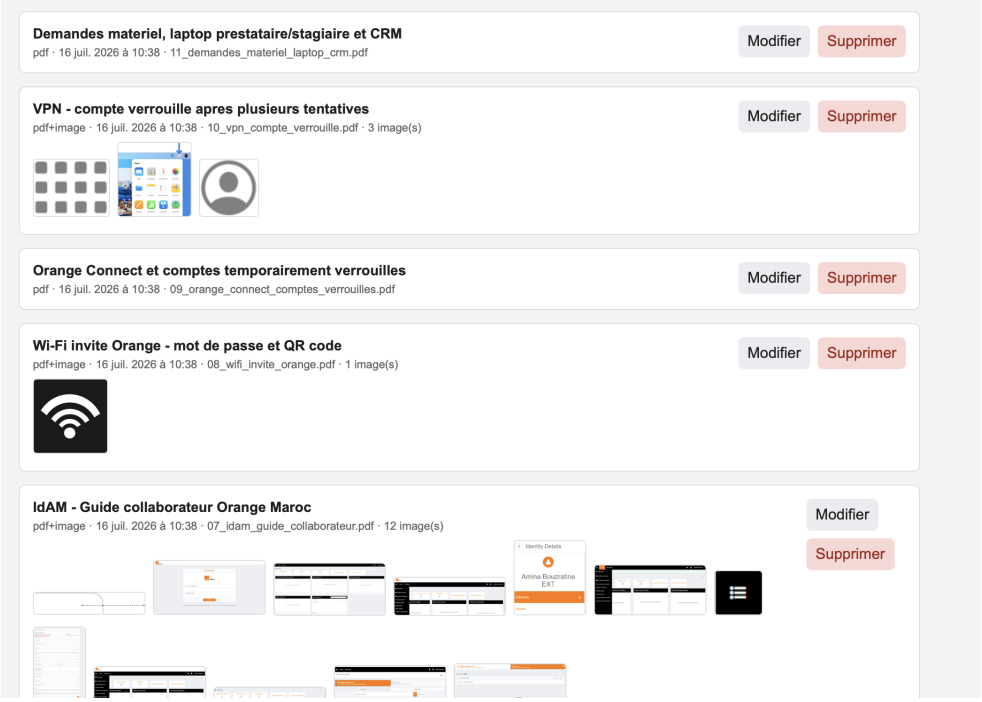


Figure 4: Knowledge management in the administration area

This area is essential because an internal chatbot is never finished. Its quality depends on the quality of its content. When users ask questions that the chatbot cannot answer correctly, the administrator can identify the missing information and improve the knowledge base.

The admin history also helps reveal repeated needs. If many users ask about the same tool or incident, it becomes a priority for documentation. The chatbot therefore becomes a feedback loop between users and support knowledge.

Adding and structuring knowledge

Adding knowledge is the first step toward improving the chatbot. The administrator gives the entry a title, writes or pastes the procedure, optionally attaches a PDF, and adds images if the procedure is visual.

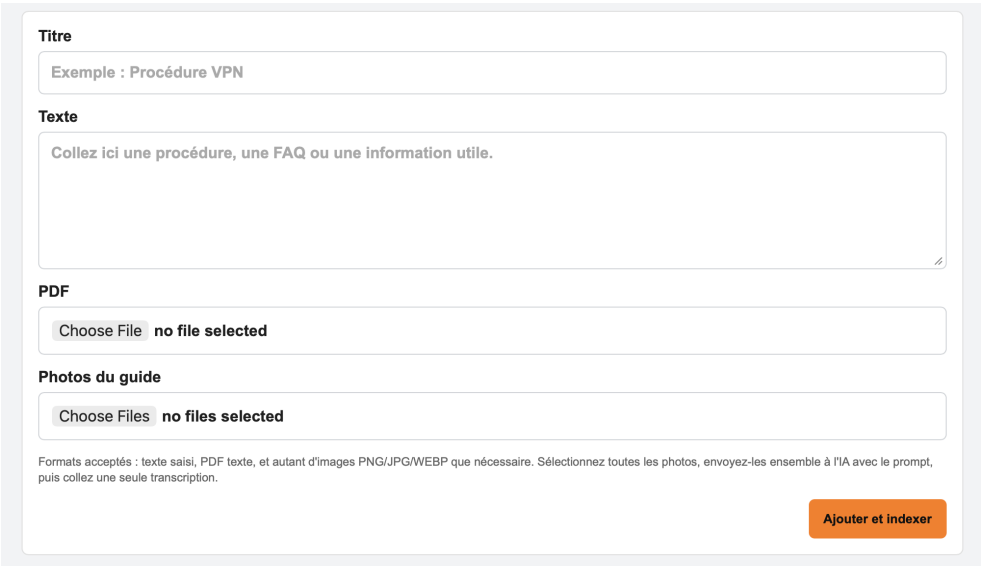


Figure 5: Knowledge creation form

The quality of the text matters. Long unstructured paragraphs are harder to retrieve and harder for users

to follow. A good procedure should have a short title, a summary, numbered steps, expected results, and conditions for escalation.

The interface can support this process by helping the administrator transform raw text into a clearer format. Once the content is structured and indexed, the chatbot can use it in future conversations. This makes the administration process directly connected to user experience.

11 Visual Guides: Concept

Visual guides are used when a text answer is not enough. Some procedures require the user to recognize a menu, a button, a setting, or an error message. In those cases, screenshots reduce ambiguity and make the answer easier to follow.

The visual guide mechanism links images to a knowledge entry. The administrator uploads the screenshots in the correct order and provides descriptions for them. When the user asks a related question, the chatbot can return both the written explanation and the associated images.

This is particularly useful for procedures such as Outlook archive configuration, Wi-Fi access, VPN steps, mobile email setup, file transfer, or identity management actions. These tasks often become clearer when the user can compare their screen with the screenshot shown by the chatbot.

The guide is not just a collection of images. It is a structured procedure: each screenshot corresponds to a specific action. This structure helps the chatbot display the right visual support at the right moment.

12 Visual Guides: Admin Workflow

The administrator creates a visual guide from the knowledge form. First, the title describes the problem or procedure. Second, the text explains the expected result and the main steps. Third, screenshots are added in the order in which the user should follow them.

The interface can generate a prompt asking an AI assistant to describe the uploaded images. The administrator sends the images and the prompt to the AI, then copies the structured transcription back into the form. This transcription explains what each screenshot shows and what action the user must take.

This workflow improves search quality. A screenshot by itself cannot be searched easily. Once it is described in text, the chatbot can associate it with keywords, actions, and user intents. The final guide becomes both visual for the user and searchable for the system.

13 Visual Guide Examples

The project includes several examples of visual procedures. These guides illustrate how visual support can help the user with concrete steps instead of a long theoretical explanation.

For an Outlook archive procedure, the chatbot can display the first screenshot to show where the user should click, then display the next screenshots as the user follows the sequence. The user does not need to guess the interface path.

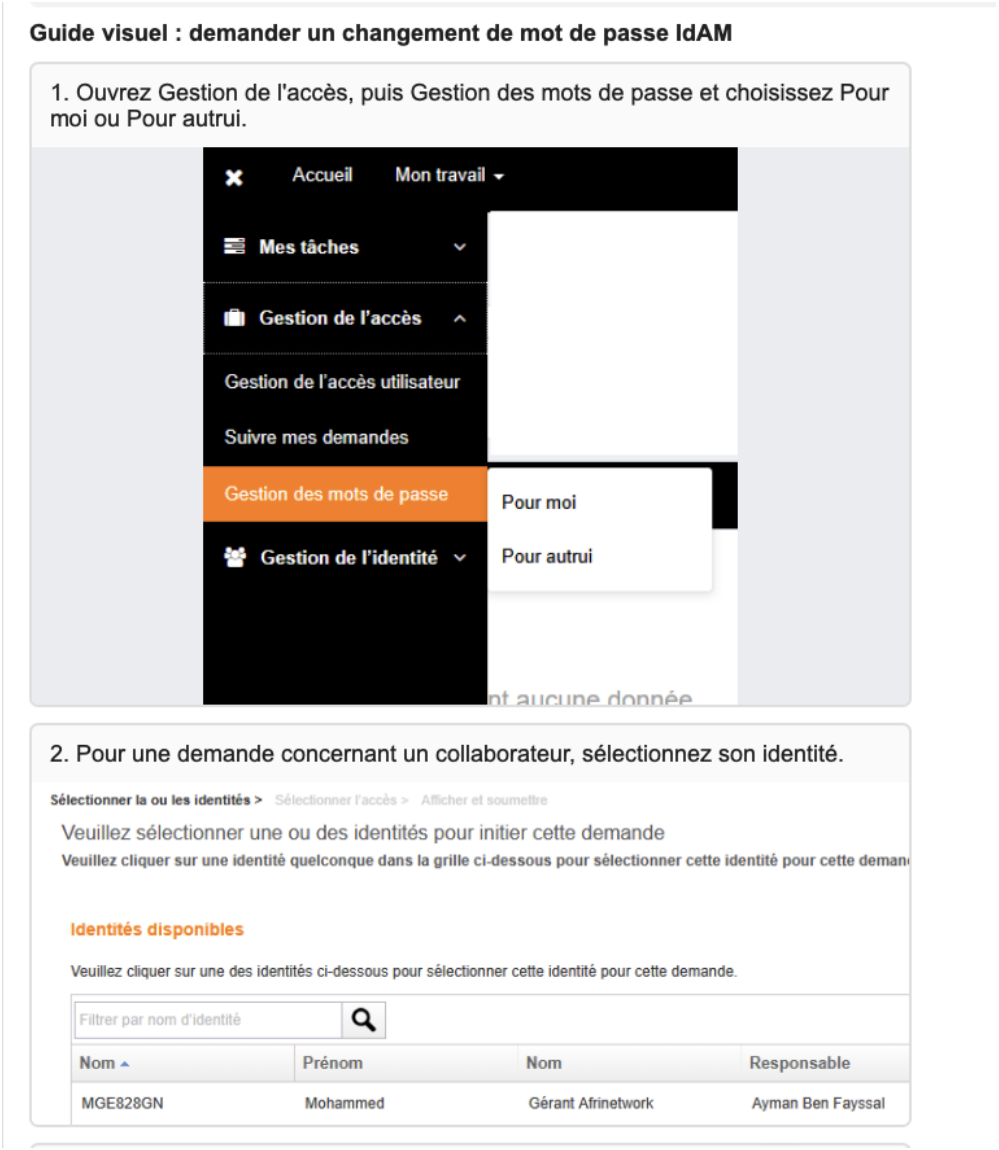


Figure 6: Visual guide for password change - Steps 1 and 2



Figure 7: Visual guide for password change - Steps 3 and 4

This type of answer is more effective than a paragraph alone. It reduces misunderstanding, especially

when the user is not familiar with the software interface. It also reduces repeated hotline calls because the same guide can help many collaborators.

Other procedure examples

The chatbot can also store guides for Wi-Fi access, identity management, file transfer, or other internal tools. These examples show why images are useful: the user can visually confirm that they are on the right screen before continuing.

In a Wi-Fi case, the user may not know which network to choose or which step comes next. A visual guide helps them follow the procedure without waiting for a support agent. If the guide does not solve the issue, the chatbot can still prepare an escalation message.

This approach turns the chatbot into a practical assistant. It does not only explain what to do; it shows the user how to do it.

14 Artificial Intelligence Role

Artificial intelligence is used as a support layer, not as the only source of truth. The chatbot first relies on internal knowledge. AI is then used when it improves the result: reformulating an answer, summarizing a procedure, analyzing a screenshot, transcribing voice, or drafting an email.

Groq can be used for fast text generation and voice transcription. Gemini can support image analysis or complementary tasks depending on configuration. The backend prepares a controlled prompt that includes the user's request, useful knowledge passages, and answer rules.

This design helps reduce the risk of generic or incorrect answers. The model is guided by internal content. If no reliable context is found, the chatbot should avoid pretending to know and instead ask for clarification or propose escalation.

AI also improves user comfort. A user can write a vague question, send an image, or dictate a problem. The system can transform these inputs into a cleaner request and produce a more understandable answer.

15 Voice, Image, and Email Assistance

The chatbot supports multiple input formats because users do not always express problems in the same way. Some users prefer writing. Others may find it easier to speak. In many support cases, a screenshot is the fastest way to show an error.

For voice, the browser records the message and sends it to the backend. The server transcribes it into text, then processes it like a normal question. This is useful when the user wants to describe a situation quickly.

For images, the user can attach a screenshot. The system can analyze visible elements and combine this information with the user's question. This helps when the user does not know the technical name of an error or interface element.

For escalation, the chatbot can prepare a HelpDesk email. Instead of leaving the user to write an unclear message, it can generate a structured description containing the context, the problem, and useful details. This improves communication with the support team.

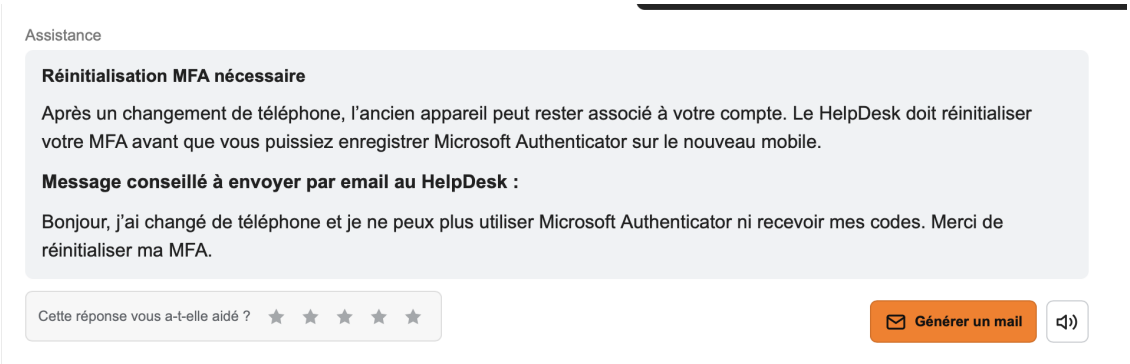


Figure 8: The chatbot prepares an escalation message



Figure 9: The generated HelpDesk email draft

16 Security and Confidentiality

Security is a central requirement in an internal support tool. The chatbot may process user questions, screenshots, and technical information. It must therefore avoid exposing sensitive data or storing unnecessary information.

API keys, passwords, tokens, and confidential data must never be placed in the knowledge base. Uploaded files should be restricted by size and format. Responses should be sanitized before display to avoid unsafe content in the browser.

The administration area must also be protected. A wrong or malicious knowledge entry could produce wrong answers for many users. Only authorized administrators should be able to add, edit, or delete content.

AI usage must be supervised as well. The system should track which functions use AI, avoid sending unnecessary confidential information, and keep a clear separation between internal knowledge and external model calls. Future deployment could include stronger authentication, network proxy rules, and stricter access controls.

17 Testing and Validation

Validation focused on realistic user journeys. The chatbot was tested with simple questions, procedure searches, knowledge creation, visual guide display, voice transcription, image support, HelpDesk email generation, and admin history review.

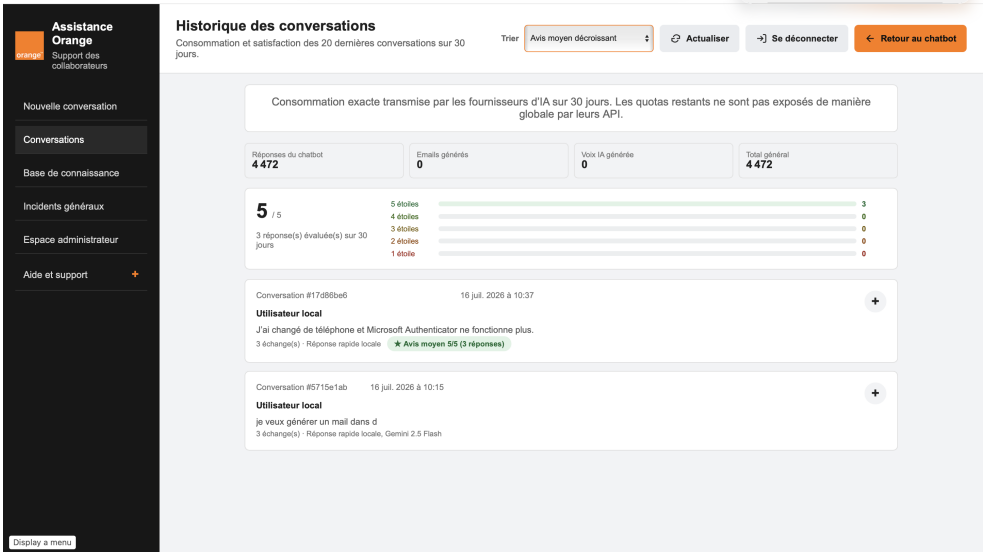


Figure 10: Conversation monitoring and user feedback

The tests showed that answer quality depends strongly on knowledge quality. A well-written procedure gives a clearer answer. A vague or incomplete procedure produces a weaker result, even if the AI model is capable.

The visual validation was also important. Screenshots must be readable, captions must be understandable, and the user must know what to do after reading the response. In a support chatbot, the final measure of quality is not only technical success. It is whether the user can solve the problem more easily.

18 Detailed Technical Stack and Justification

The technological stack was chosen to maximize rapid development, performance, and ease of deployment on an enterprise Windows Server environment. The choices reflect a balance between modern asynchronous capabilities and robust traditional architectures.

Backend: Python and FastAPI

The core backend is built using Python 3.11+ and FastAPI. FastAPI was selected for its high performance, native support for asynchronous programming (async/await), and automatic generation of OpenAPI documentation. Since the chatbot must handle multiple concurrent requests—such as searching the database, calling external AI APIs (Gemini and Groq), and serving static files—an asynchronous framework prevents the server from blocking during I/O operations. Uvicorn is used as the ASGI server to run the FastAPI application, providing a fast and lightweight execution environment.

Frontend: HTML, CSS, and Vue 3

To minimize build complexity and avoid managing a separate frontend compilation pipeline (like Node.js or Webpack), the user interface is built using standard HTML5 and CSS3, enhanced with Vue 3 included directly via a script tag. This approach allows the frontend to remain a set of static files served by FastAPI (`index.html`, `historique.html`, `admin-login.html`). Vue 3 provides the necessary reactivity to manage chat state, user inputs, and asynchronous UI updates without the overhead of a Heavy Single Page Application (SPA) architecture.

Database: SQLite and MySQL

The application supports dual database engines. During local development and testing, SQLite is used automatically if no other configuration is provided. It stores all data in a local `chat_logs.db` file, which requires zero configuration. For the enterprise production environment, the system connects to a MySQL 8 server (with Azure SQL support available as a fallback in the codebase). MySQL provides the concurrency, reliability, and security required for an internal support tool. The application handles its own database schema creation at startup, ensuring that tables are automatically provisioned.

AI Providers: Gemini and Groq

The chatbot does not rely on a single AI provider, avoiding vendor lock-in and ensuring high availability. It integrates Google's Gemini API (using models like `gemini-2.5-flash`) for its strong reasoning and multimodal capabilities, including processing user-uploaded images and generating structured responses. Groq is integrated as an alternative or primary provider for text generation and voice transcription (using `whisper-large-v3-turbo`), chosen for its extremely low latency and high-speed inference. The system implements a robust model fallback mechanism that can dynamically switch between these providers if one becomes unresponsive.

19 In-Depth System Architecture

The architecture of the Orange Helpdesk Chatbot is designed to strictly control the flow of information. It enforces a Retrieval-Augmented Generation (RAG) pattern where the AI is constrained by the internal knowledge base.

The Chat API Flow

When a user submits a question via the `/api/chat` endpoint, the backend performs a series of orchestrated steps:

1. **Input Processing:** The user's input (which can be text, an audio recording, or an image) is received. If an audio file is provided, the backend sends it to Groq's Whisper model for immediate transcription. If an image is provided, it is saved locally in the `chat-images/` directory and prepared for multimodal analysis by Gemini.
2. **Knowledge Retrieval:** The function `retrieve_knowledge()` searches through both the static `knowledge.txt` file and dynamic entries in the database. It scores documents based on keyword matching and relevance to the user's query, extracting the most pertinent paragraphs.
3. **Prompt Construction:** The backend constructs a highly structured prompt in `build_helpdesk_prompt()`. This prompt includes strict system instructions (e.g., "You are an internal IT support assistant for Orange"), the exact passages retrieved from the knowledge base, the user's specific question, and any conversational history. It forces the AI to reply in a specific JSON format containing the HTML response, a boolean flag recommending HelpDesk escalation, and any associated visual tutorial IDs.
4. **Model Execution and Fallback:** The `ask_with_model_fallback()` function attempts to call the primary configured AI model. If the primary model fails or times out, it automatically catches the exception and retries the request using the next model in the fallback list (e.g., switching from Groq's Llama-3 to Gemini).
5. **Response Sanitization:** The JSON response is parsed and validated. The generated HTML is sanitized to prevent Cross-Site Scripting (XSS) attacks before being returned to the user interface.

Multimodal Capabilities

The architecture uniquely supports multimodality. When an employee encounters an error message on their screen, they can capture a screenshot and upload it directly to the chat. The backend routes this image to the Gemini vision model along with the user's text. The AI analyzes the interface elements and text within the image, combining it with the internal procedures retrieved from the database, to provide a highly contextualized troubleshooting step.

20 Database Design and Schema

The application's data persistence layer is essential for both functionality and administration. The schema is designed to track usage, preserve knowledge, and facilitate continuous improvement.

Core Tables

- **`chat_logs`:** This is the primary table recording user interactions. It stores the user's query, the assistant's reply, the specific AI model used, response latency, token usage, and user satisfaction ratings (thumbs up/down). It also records references to any uploaded images (stored in `chat-images/`) and visual tutorials displayed during the interaction.
- **`knowledge_documents`:** This table holds the dynamic knowledge base managed by administrators. Each entry includes a title, content, associated keywords, and optional visual tutorial configurations. When a PDF is imported, its extracted text is stored here, making it instantly available to the `retrieve_knowledge()` function.
- **`frequent_questions`:** To optimize performance and provide instant answers to the most common queries, administrators can define frequent questions. These bypass the AI generation entirely, returning pre-approved HTML answers instantly.
- **`service_alerts`:** This table manages active IT service alerts (e.g., "VPN maintenance at 22:00"). Active alerts are fetched by the frontend on load and displayed prominently to reduce redundant support queries during known outages.
- **`ai_usage_events`:** Detailed telemetry regarding AI API usage is logged here. It tracks the provider, model, prompt tokens, completion tokens, and the feature that triggered the call (e.g., chat, transcription, formatting). This is vital for cost monitoring and capacity planning.

Data Migration Strategy

Recognizing that the application might be prototyped on SQLite and later moved to MySQL, a dedicated migration script (`scripts/migrate_sqlite_to_mysql.py`) was developed. This script carefully connects to both databases simultaneously, reads all records from the local `chat_logs.db`, and precisely inserts them into the production MySQL instance. This ensures that no conversation history or admin configurations are lost during the transition from development to production.

21 Frontend and Administration Mechanics

The user-facing and administrative interfaces are designed to be intuitive, fast, and require minimal client-side processing.

The Chat Interface (`index.html`)

The main chat interface leverages Vue 3 for reactive DOM updates. Key features include:

- **Dynamic Message Rendering:** Messages are rendered dynamically. User messages appear on the right, while assistant messages (rendered as safe HTML) appear on the left.
- **Voice Recording Integration:** Using the browser's `MediaRecorder` API, users can record audio. The blob is sent to the backend as a multipart form data request, transcribed, and injected into the chat flow seamlessly.
- **HelpDesk Email Generation:** If the AI determines that an issue requires human escalation, the frontend displays an "Escalate to HelpDesk" button. Clicking this triggers an endpoint that drafts a structured email (using a `mailto:` link) containing the conversation context, saving the user from rewriting their problem.

The Administration Panel (`historique.html`)

The administration panel is a critical component for the lifecycle of the chatbot, accessible only via a secure `ADMIN_HISTORY_KEY`.

- **History Monitoring:** Administrators can view a paginated list of all conversations, filtering by date or satisfaction rating. This allows the support team to identify where the chatbot failed to provide a correct answer.
- **Knowledge Management:** Through the UI, admins can create, edit, or delete knowledge documents. They can also trigger the batch import of prepared PDFs from the `knowledge_base_uploads/manifes` file.
- **AI Formatting Support:** When an admin is writing a new procedure, they can click an "AI Format" button. This sends their rough notes to the AI, which reformats the text into clear, numbered steps with a standardized structure, ensuring high-quality knowledge entries.

22 Deployment and Infrastructure

Deploying an internal enterprise application requires specific adaptations for Windows Server environments, security policies, and internal network constraints.

IIS and Reverse Proxy Configuration

The application is hosted on a Windows Server utilizing Internet Information Services (IIS). Since IIS cannot run Python natively in a highly performant asynchronous manner, the FastAPI application runs via Uvicorn on a local loopback port (e.g., `127.0.0.1:8001`). IIS is configured as a reverse proxy using the URL Rewrite module. The `web.config` file intercepts incoming HTTPS traffic on port 443 and securely routes it to the local Uvicorn instance. This topology ensures that Uvicorn is never directly exposed to the enterprise network, leveraging IIS for SSL termination, request filtering, and logging.

Process Management

To manage the Uvicorn process, a series of batch scripts (`Start Chatbot Windows.bat`, `Stop Chatbot Windows.bat`) are provided. These scripts activate the Python virtual environment and launch the server. In a full production setup, these scripts can be wrapped in a Windows Service (using tools like NSSM) to ensure the chatbot automatically starts upon server reboot and restarts if it crashes.

Enterprise Proxy Handling

Enterprise networks often restrict direct outbound internet access. Since the chatbot must communicate with external AI providers (Groq and Gemini), the backend is configured to respect standard proxy

environment variables (`HTTP_PROXY`, `HTTPS_PROXY`, `NO_PROXY`). The `httpx` client used within the backend automatically routes API requests through the corporate proxy, ensuring compliance with internal security policies.

23 Security and Maintenance

Maintaining the integrity, privacy, and security of internal support conversations is a top priority.

Security Best Practices

- **Environment Variables:** Sensitive information, such as database passwords (`MYSQL_PASSWORD`) and API keys (`GEMINI_API_KEY`), are strictly isolated in a `.env` file. This file is explicitly ignored by Git (`.gitignore`) to prevent accidental leaks into the source code repository.
- **Sanitization:** All HTML generated by the AI is treated as untrusted. The backend utilizes HTML sanitization libraries to strip out potentially malicious scripts before the frontend renders the response, preventing Cross-Site Scripting (XSS).
- **Admin Authentication:** Access to the administration APIs and frontend is protected by the `ADMIN_HISTORY_KEY`. Requests must include this key in the headers or cookies to be authorized.

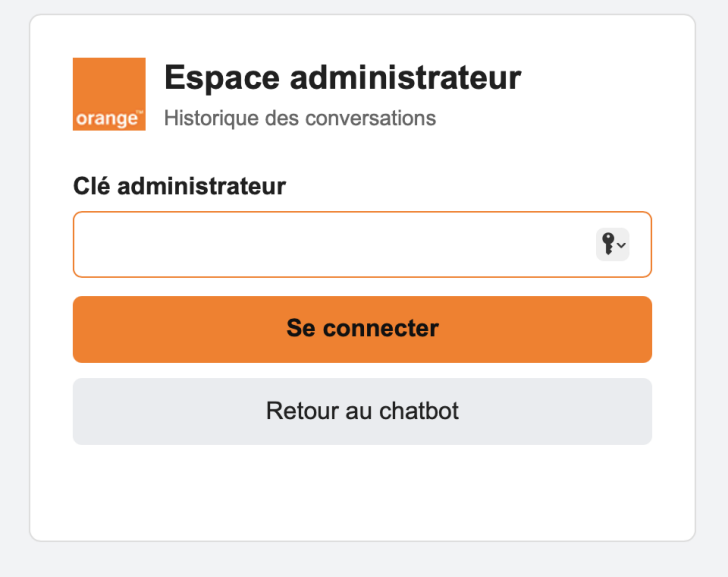


Figure 11: Administrator authentication interface

Maintenance and Backup Strategy

As outlined in the `HANDOVER.md` document, regular maintenance is required to ensure smooth operation. The runtime storage directories—such as `chat-images/` for user screenshots and `tts-audio-cache/` for generated audio—are excluded from version control but must be included in the server's daily backup routine. The MySQL database should be backed up nightly. Furthermore, the knowledge base must be continuously updated by administrators based on the analytics gathered from the `chat_logs` table to prevent the chatbot's answers from becoming obsolete. The prototype covers the main expected workflows: asking questions, receiving structured answers, using internal knowledge, adding content through the admin interface, displaying visual guides, handling screenshots, transcribing voice, and preparing HelpDesk emails.

The project also proves the importance of administration. A chatbot without content maintenance becomes outdated. A chatbot with an active knowledge base can improve over time and respond to real user needs.

Some limits remain. The quality of answers depends on the available procedures. Integration with internal systems can be improved. Authentication and access control should be strengthened for production. AI calls should be monitored carefully, especially in an enterprise context.

Future improvements may include detailed analytics, better ranking of knowledge entries, automatic detection of repeated issues, stronger identity integration, a local LLM proxy, and richer admin dashboards. These improvements would make the solution more robust and easier to manage at scale.

Personal review

This internship allowed me to work on a complete project: needs analysis, backend development, frontend design, knowledge management, AI integration, visual-guide workflow, testing, and security reflection. It helped me understand how a technical solution must answer a real operational need.

I learned that a chatbot is not only a conversation interface. It is a system built around content quality, user experience, backend orchestration, and continuous improvement. The most important challenge is not only generating an answer, but generating the right answer from the right internal source.

The developed chatbot addresses the initial need by helping collaborators obtain quick assistance and reducing repetitive support requests. It provides a solid foundation for an internal assistant that can evolve with new procedures, new integrations, and better monitoring.

In conclusion, the project demonstrates that combining a structured knowledge base with controlled AI features can improve internal support. The value of the solution will continue to grow if the knowledge base is maintained, user feedback is analyzed, and the chatbot is progressively integrated into the company's digital environment.